

A Hybrid Approach to Quantitative Software Project Scheduling Within Agile Frameworks

Michael Jahr, *European University of Applied Sciences (EUFH), Department for Industrial Management, Neuss, Germany*

ABSTRACT ■

In this article we focus on the quantitative project scheduling problem in IT companies that apply the agile management philosophy and Scrum, in particular. We combine mathematical programming with an agile project flow using a modified multi-mode resource constrained project scheduling model for software projects (MRCPS^{SP}). The proposed approach can be used to generate schedules as benchmarks for agile development iterations. Computational experiments based on real project data indicate that this approach significantly reduces the project cycle time. The approach can be a useful addition to agile project management, especially for software projects with predefined deadlines and budgets.

KEYWORDS: software project scheduling; quantitative project management; agile software development; Scrum

INTRODUCTION ■

Developing individual software applications in complex projects is a multi-layered task. In projects with predefined deadlines and budgets, managers must oversee delivery dates and control project costs. Traditional production industries (e.g., in the automotive sector or the metalworking industries) can make use of quantitative methods and models to support project management. In this article we combine quantitative scheduling with an agile project flow called Scrum (Martin, 2002), creating a hybrid approach. Scrum is an agile project management concept that focuses on working software rather than creating strict schedules. Based on a real-life case, we present a modified multi-mode resource constrained project scheduling model (MRCPS^{SP}) and add it to a given Scrum flow. Practical tests have shown that using the hybrid approach with a modified process flow chart significantly improved the adherence to deadlines and budgets. Therewith we show that agile methods and traditional approaches are not contrary ways, and that quantitative models are a useful support to project management in agile frameworks. The structure of the article is as follows: The second section briefly deals with some fundamentals of traditional planning methods and the agile methodology; thereby, possible connections between both approaches are focused on. Furthermore, the real-world case of an IT company that manages its projects with Scrum is described. The company's actual scheduling problems within the used Scrum environment, as well as the aims and scope, are highlighted. The third and fourth sections deal with the used quantitative methodology and present a hybrid approach to combining quantitative scheduling and agile frameworks. The hybrid Scrum flow is tested based on real data in the fifth section and assessed in the sixth section; finally, the seventh section provides a summary and recommendations for further research.

The Quantitative Software Project Scheduling Problem

Traditional Planning Versus Agile Project Management

In traditional industries (e.g., the automotive sector), planning and the resulting plans are process oriented and aimed at sequencing the necessary operations. The focus lies on standardized operations that try to eliminate procedure uncertainties. The corresponding planning methods have developed over many decades. As a result, there are efficient operational research methods and models available (Steinrücke, 2011; Steinrücke & Jahr, 2012), which can be applied to generate, for example, effective production plans. Compared with traditional industries (e.g., metal working and automotive), the IT industry is a younger industry. Hence, different approaches to organizing work and jobs have emerged to challenge the traditional plan-driven ways; therefore, each

A Hybrid Approach to Quantitative Software Project Scheduling Within Agile Frameworks

individual application is programmed within a distinct software project. The basis for a flexible project management approach is the *Agile Manifesto* (Agile Alliance, 2001; Nachamai, Senthil Vadivu, & Tapaskar, 2011; Jalali & Wohlin, 2011). Its goal is to develop the most practicable solution that fulfills the actual customer's needs rather than the possible best result (Frank Cervone, 2011; Takeuchi & Nonaka, 1986). An established agile project management concept is Scrum (Schwaber, 2004; Reifer, 2002). Salo and Abrahamsson (2008), for example, surveyed the use of agile software development methods and Scrum in European industrial organizations. Their results indicate that the appreciation of the agile methods and their individual practices appears to increase once adopted and that Scrum is used rather systematically. Under these circumstances, traditional and agile methods seem to be contrary ways; however, software projects with defined specifications and contracted deadlines, as well as budgets, thus face strict targets. Here, most notably quantitative management support can complement agile methods and contribute to meeting deadlines and budgets. In the next section, we describe a real-life case of an IT company using Scrum to manage strictly targeted projects. Subsequently, we present a variation of the Scrum process, including quantitative scheduling.

Case Study of an IT Company Using Scrum

In the considered case, we analyzed the software project scheduling process and the project management in an enterprise that develops individual software programs in the finance sector. The company was selected because it is one of the market leaders in the field of individualized banking applications and, furthermore, represents a forerunner in modern agile project management in its segment. The company focused on sales and administration applications in its product portfolio. In principle, the company distinguished between small projects (redevelopments of existing

applications) and large projects (new software applications up to entire platforms); for the software projects in the considered company a modified version of Scrum is used. The enterprise occasionally mandated a specialized IT consultancy that provided Scrum coaches to support the process flow and respond to organizational questions. There are three roles in the Scrum project process (Schwaber, 2004): The product owner, who has a business background and operates as a project manager, including responsibility for deadlines, budgets, and project teams. Next to the product owner is the ScrumMaster, a software programmer, who serves as a link to the Scrum team, which consists of the software developers and encompasses a team of up to 15 members. The ScrumMaster has a supporting and administrative role. In contrast to the company's five product owners and five ScrumMasters, the Scrum teams were mostly rearranged for each new project. Two types of software developers are distinguished. One small group of 15 people is built by the senior programmers with longtime professional experience. The larger group of up to 40 people includes the regular programmers. A software application consists of several functional modules, so-called User Stories, which contain descriptions of the desired specifications. One or more User Stories are developed in single closed cycles (iterations or sprints) in which the Scrum team works without external influence. At the end of the iteration, the team presents the results to the product owner; hence, the iteration time boxes are fixed, whereas the scope is difficult to predict.

Deficiencies in the Project Management Process

To analyze the project management process we attended the iteration planning meetings, short daily meetings, and the retrospectives of five different projects, which were handled simultaneously by different project teams over a time period of three months. During this time we conducted informal interviews with

all product owners, ScrumMasters, and senior developers because these people have the most expertise. Our main questions focused on the adherence to delivery dates and the relations between costs and budgets. After the analysis, we were able to outline a typical project run, which is exemplified in the following example. A national finance company ordered a new application consisting of three modules (i.e., User Stories): client administration, cross-selling management, and order management. Next to a product owner and ScrumMaster, a Scrum team consisting of seven members (two senior and five regular developers) was established. The desired specifications were analyzed by the entire project team and a project duration period of 16 weeks was estimated, which already included a two-week time buffer. A corresponding delivery date for the last module was stipulated with the customer, and the project budget was calculated based on the project team size (nine people), the estimated project duration (five work days per week), and an average daily cost rate for personnel of €1,500 (US\$2,100), totaling up to €1,080,000 (US\$1,512,000). User Stories one and two should be developed in two iterations of six weeks each, and the last User Story in a single iteration of four weeks. The iterations were intended to be closed periods (sprints) in which the product owner may not influence the Scrum team. Short daily meetings contained brief feedback sessions, mostly addressing technical questions. After iteration one, roughly one third of the first module was completed due to unexpected complexity; thus, next to the second User Story, the unfinished first User Story also had to be programmed into iteration two. So, in the end, neither of the User Stories was completed after iterations two and three. At the stipulated delivery date, no module was delivered. In the meantime, the Scrum team was split up to work on several User Stories simultaneously, which increased the problems. Last, a second Scrum team of five additional

regular developers was added. The project was finalized with a project duration period of 34 weeks (213% of the estimated duration). The project costs totaled €2.89 million (US\$4,046,000) (268% of the estimated costs). Based on the described project and the interviews, we could determine that the major problems were massively overrun deadlines and project costs, which consistently escalated irrespective of the project type. Project teams systematically underestimated project durations and costs. The Scrum flow normally contains several elements to rethink the operating cycles and to learn from completed iterations (e.g., the iteration review and the retrospective; see later). The senior developers indicated that the Scrum teams had difficulties estimating the project parameters (times, complexity), because each project was individual without standardization. They concluded that within the complex development phases, the overview, including deadlines and scopes, was often lost; therefore, the project management was in need of an effective tool to benchmark the iteration cycles and to estimate the project costs. The company management claimed four central requirements:

1. Delivery dates must be met.
2. Project budgets must be met.
3. Adherence to agile methods and to the basic Scrum process.
4. Simple processes and methods applicable to small and large projects.

In the following sections we present a hybrid approach to extending the existing Scrum flow by using a quantitative scheduling approach to supporting the project teams.

Modeling Software Projects With a Modified Resource Constrained Project Scheduling Approach

Quantitative Software Project Modeling

Our primary goal was to support software project management with objec-

tive schedules, which can be used as a benchmark or basis for the project phases (iterations). These schedules must enable resource allocation concerning (in-)experienced software developers and delivery cycle times. Both aspects are needed to meet deadlines, control costs, and calculate product prices respectively. Therefore, we decided to depict the software development process with an Activity-on-the-Node-Graph (AoNG, see next section) in order to apply a mathematical optimization approach to determine the desired objective quantitative schedule (see later). Thus, the AoNG is the basis for quantitative scheduling and the central element of our approach. From the process point of view, the (senior) computer programmers are the key factors in the actual software development process rather than the additional agile roles (i.e., product owners and Scrum-Masters); hence, the AoNG must fulfill two requirements: First, it must be created focusing on all potential activities involving software developers. Second, it must be flexible enough so that separate activities can be removed in case they are not necessary for a specific

iteration. The latter can be achieved by simple data adjustment (see later).

Depicting the Development Process With an Activity-on-the-Node-Graph

With the AoNG, we aim for an accurate depiction of the specific software development process (Figure 1). The boxes represent the activities and the arrows represent the activity orders that lead to predecessor and successor relations, which at first counteract the continuous agile software development philosophy, but we can then use this approach later on to extend the agile framework and support project management (see later). To improve an AoNG thus becomes part of the extended agile project management framework creating a hybrid approach (Figure 2). We present the AoNG, which was created by the Scrum team and supported and examined by the Scrum coaches as described in the case study from the previous section. The developed basic AoNG includes 15 activities; here, all activities were considered that (possibly) involved software developers (i.e., all activities consuming available software programmer time and capacity). This approach

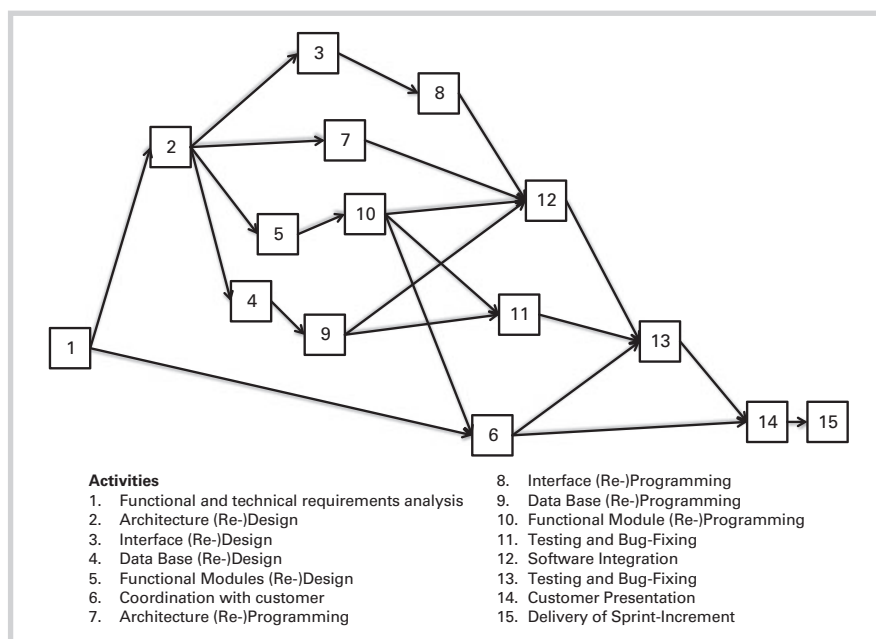


Figure 1: Activity-on-the-node-Graph (AoNG) for activities in the Scrum sprints.

A Hybrid Approach to Quantitative Software Project Scheduling Within Agile Frameworks

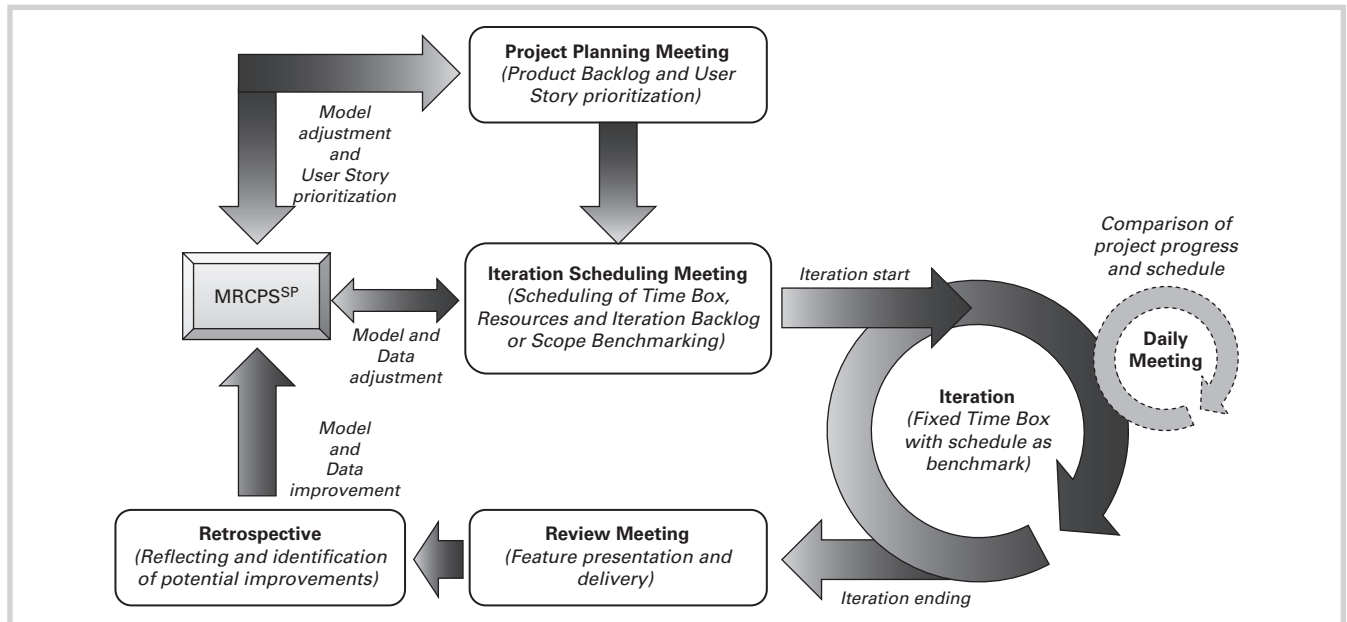


Figure 2: Hybrid quantitative software project scheduling.

proved to be the best practice in projects that had to meet specific deadlines.

First, the project team analyzes the product requirements to be developed in the following iteration (activity 1). By data adjustment, activity 1 can be ignored and thus shifted to a pre-iteration planning; then, the affected software components were designed or redesigned if already existing components or old products had been addressed (activities 2 through 5). Afterward, the product concepts should be reviewed with the customer before development (activity 6). Software represents a complex product, so at least one software programmer was always involved as the expert in each customer interaction and hence missed the actual programming stage. Moreover, this activity can be used as an additional time buffer; software development takes up the major part (activities 7 through 10). A first testing phase (activity 11) is used to test the single modules and to prepare the software integration (activity 12). The second testing phase (activity 13) aims at a functional iteration increment, which is then presented to the customer (activity 14). Finally, the

iteration increment is delivered (activity 15). As explained earlier, we explicitly incorporated coordination activities that can function as time buffers (activities 6, 11, 14, and 15) but can also be excluded from consideration. By deviating from this AoNG, other processes can be depicted, which in general, are independent from the mathematical optimization model presented in the next section.

Multi-Mode Resource Constraint Project Scheduling for Software Projects With Time Lags and Non-Constant Resources (MRCPS^{SP})

For quantitative scheduling we decided to make use of approved operations research methodology. Specifically, we aimed at implementing a modified multi-mode resource constraint scheduling model for software projects (MRCPS^{SP}) in order to schedule, for example, the derived AoNG described earlier. The resource constrained project scheduling problem (RCPSP) is one of the basic problems in production planning and has already been intensively discussed (Brucker, Drexler, Möhring, Neumann, & Pesch, 1999). For further

readings please refer to Drexler and Gruenewald (1993), Brucker (2002), Horbach (2010), and Behrouz, Hojjat, and Esmaeil (2012). Due to the considered problem structure in software projects we applied the multi-mode type with time lags and non-constant resources. To use the MRCPS^{SP} for the quantitative scheduling, we focus $i=1, \dots, N$ activities. Thereby S_j represents all successors j of job i . The model formulation allows for a random number of modes, which can represent any kind of specialized employee. In the considered case study, two types of software developers were distinguished (i.e., regular inexperienced software programmers and experienced senior software programmers, which formed the minor group). Each activity can be executed by one of these software developer types, which is depicted by modes $m_i = 1, \dots, M_i$. To perform activity i in mode m_i takes p_{i,m_i} time units. A maximum iteration cycle time, which is needed for computation as an upper limit, is given by T and the maximum project duration is determined by PD . By $\frac{PD}{T}$ the number of iterations can be estimated. The actual iteration cycle times and the actual

project duration as accumulated iteration cycle times are computed by the approach and thus are a central result and benefit. The software developers are further represented by non-constant renewable resources $k=1, \dots, K^p(t)$. The estimation of software developer capacity is an intricate problem. To the best of our knowledge, there is no completely satisfying approach available, so that there is a tendency to neglect any kind of quantitative capacity evaluation. In fact, the proposed model can be formulated as unconstrained version. Nevertheless, we are convinced that any unconstrained scheduling violates realistic management; so, we borrowed an interesting element from the agile framework (i.e., story points) and used them in a derived form for the model formulation. Later we explain this aspect in more detail. Next to capacitated renewable resources (manpower) the project budget is also restricted (non-renewable resource out of R^v). The iteration budget is determined by $\frac{\text{Project price}}{\text{Number of iterations}}$. A feasible schedule must consider both the available

developer capacities and the available iteration budget. Clearly, if all single iteration schedules are feasible, then the project can be expected to meet the deadline and the total project budget. We use this simple appearing idea as a guideline. All activities consume the available (non-)renewable resources. In Table 1 the notation used for the MRCPS^{SP} is summarized.

The MRCPS^{SP} can be formulated as follows:

$$\text{Minimize } Z = \sum_{m_i=1}^{M_i} \sum_{t=EF_i}^{LF_i} t \cdot x_{i,m_i,t} \quad (1)$$

$$\text{s.t.} \\ \sum_{m_i=1}^{M_i} \sum_{t=EF_i}^{LF_i} x_{i,m_i,t} = 1 \forall i \in N \quad (2)$$

$$\sum_{m_i=1}^{M_i} \sum_{t=EF_i}^{LF_i} t \cdot x_{i,m_i,t} \leq \sum_{m_i=1}^{M_i} \sum_{t=EF_i}^{LF_i} (t - p_{i,m_i} - d_{i,j}^{\min}) \cdot x_{i,m_i,t} \\ \forall i \in N, \forall j \in S_j \quad (3)$$

$$\sum_{i=1}^N \sum_{m_i=1}^{M_i} \sum_{k=1}^{K^p(t)} \sum_{\tau=\max\{t, EF_j\}}^{\min\{t+p(i,m)-1, LF_j\}} x_{i,m_i,\tau} \leq C_{k,m}^p \quad \forall m_i \in M_i, \\ \forall k \in K^p(t), \forall t \in T \quad (4)$$

$$\sum_{i=1}^N \sum_{m_i=1}^{M_i} \sum_{f=1}^{F_i} \sum_{t=EF_i}^{LF_i} x_{i,m_i,t} \leq C_f^v \quad \forall f \in R^v \quad (5)$$

$$x_{i,m_i,t} \in (0,1) \quad \forall i \in N, \forall m_i \in M_i, \\ t = EF_i, \dots, LF_i \quad (6)$$

In the objective function (Equation 1) the iteration cycle time is minimized; next to this, different objectives are possible. For example, the scheduling could aim at minimized iteration costs, which could lead to longer iterations. Here, the deadlines were of special importance so that it was preferred to strain the budget in favor of the duration. The chosen model formulation requires that the earliest and latest start times, as well as the finish times, are calculated by simple forward and backward recursion in advance, using the shortest durations for all jobs (Alvarez-Valdes & Tamarit, 1989). This procedure is known from the uncapacitated precedence diagram method. Constraint (Equation 2) is the completion restriction ensuring that each job is executed using exactly one mode (i.e., senior or regular software developer). Through constraints (Equation 3) the precedence relations between the activities are determined; thereby, minimum time lags $d_{i,j}^{\min}$ can be set and these can be used to incorporate an additional time buffer between two activities. In constraints (Equation 4) and (Equation 5), the capacities for (non-)renewable resources are provided; finally, constraint (Equation 6) indicates the binary restrictions. The model presented allocates the available software developers to activities and considers the capacities and budget. The result is a schedule that indicates the activity finish time and which developer type (senior or regular software developer) will execute which activity. Therewith, important assistance is rendered. The project manager is given clear indication for resource planning, and the project teams receive a feasible schedule they can rely on.

Measuring Software Code Programming With a Three-Step Function Point System

The scheduling model presented assumes that the personnel resources

Indices	
N	Set of activities
M_i	Set of modes for activity $i \in N$ representing skill levels with $m_i \in M_i$
$K^p(t)$	Set of non-constant renewable resources representing manpower with $k \in K^p(t)$
R^v	Set of non-renewable resources representing budget with $f \in R^v$
S_j	Set of successors $j \in N$ of job $i \in N$
T	Planning horizon representing the maximum sprint duration
Parameters	
$C_{k,m}^p$	Available mode dependent capacity of renewable resource $k \in K^p(t)$ (available manpower)
C_f^v	Available capacity of non-renewable resource $f \in R^v$ (available sprint budget)
$d_{i,j}^{\min}$	Minimum time lag between activity $i \in N$ and successor activity $j \in N$
EF_i	Earliest finish time of job $i \in N$ calculated by uncapacitated critical path analysis
LF_i	Latest finish time of job $i \in N$ calculated by uncapacitated critical path analysis
p_{i,m_i}	Duration of activity $i \in N$ executed with mode (skill level) $m_i \in M_i$
PD	Maximum project duration
$\lambda_{i,m_i,k}^p$	Usage of renewable resource $k \in K^p(t)$ by activity $i \in N$ with mode $m_i \in M_i$
$\lambda_{i,m_i,f}^v$	Usage of non-renewable resource $f \in R^v$ by activity $i \in N$ with mode $m_i \in M_i$
Variables	
$x_{i,m_i,t}$	Binary variable indicating whether activity $i \in N$ is finished in time period $t \in T$ with mode $m_i \in M_i$ ($x_{i,m_i,t} = 1$) or not ($x_{i,m_i,t} = 0$)

Table 1: Notation MRCPS^{SP}.

A Hybrid Approach to Quantitative Software Project Scheduling Within Agile Frameworks

used are capacitated (renewable resources), which correctly depicts the fact that employees can only deliver a specific performance during their working hours. Whereas in production industries these performances can be measured in specific quantity units or time units, measuring software programmer performance (e.g., software code) is more complex, especially in agile frameworks where programming teams work in given fixed time boxes. Without external interference, results are presented at the end of the iteration; in software projects with fixed deadlines and limited budgets, however, the cycle times and costs must be appraisable to some degree for the project manager; therefore, performances must be assessed. Agile frameworks, and notably Scrum, include a so-called “function point system,” which is a dimensionless evaluating system (Albrecht, 1979; Abrahao, Poels, & Pastor, 2006). In general, these function points or story points represent a relative estimation independent of the expertise and productivity of the individual developer; hence, using these elements for capacity evaluation is an inaccuracy. But, we make use of the fact that Scrum teams are familiar with evaluating software modules by function points and transfer this mechanism on activities as defined in the AoNG. Thus, two aspects are assumed. First, software developers have a specific available capacity on a daily base, whereby senior and regular developers are assigned the same amount. Second, even though the developers possess the same capacity, each activity complexity and difficulty differs for senior or regular developers consuming different capacity quantities. This is explained by varying experience levels. For example, programming software code for business applications (e.g., finance algorithms) is less complex for a senior developer due to a certain degree of experience. A regular developer must be worked in the business topics. Within the pure Scrum process of the considered company, the activity

complexity was measured in a reduplication scheme. Because the software developers were used to this method, we agreed to adopt this procedure, even though deviating methods (e.g., exponential style data) possibly would have been more common in other industries. So, each activity is evaluated in a three-step function points system, with 4 points assigned for simple activities, 8 points for complex activities, and 16 points for extremely complex activities. For example, an activity A can be simple to solve for a senior programmer and also for a regular programmer, so that the specific activity A is evaluated with 4 points to indicate the complexity for a senior programmer and 4 points to indicate the complexity for a regular programmer. However, an activity B can be simple to solve for a senior programmer but complex for a regular programmer. Therefore, the activity B is evaluated with 4 points for senior programmers and 8 points for regular programmers. This approach is basically an arbitrary assumption, but the practical tests showed very good results (see later). Measuring the software programming activities aiming at the best accuracy is an element for further research. The proposed modeling is not limited to this approach and is flexible with regard to the measuring methodology.

Integrating Quantitative Project Scheduling Into an Agile Project Management Framework

A Hybrid Approach for Quantitative Software Project Management

In the previous sections, a quantitative scheduling model was presented to support software project management. These kinds of models typically refer to operation sequencing problems. The agile management concept works differently; here, iteration time boxes are fixed and the scope is intended to be variable. However, objective quantitative schedules can operate as benchmarks and decision bases because they

offer reliable cycle times to schedule deadlines and cost information for budget planning. As a consequence, the MRCPS^{SP} must be integrated into the agile method, for example, Scrum (Figure 2). We therefore vary the Scrum process. The agile methodology is hence extended by “traditional” quantitative scheduling approaches.

Thereby, with the MRCPS^{SP} model an additional flow is added to the regular Scrum elements (i.e., Review, Retrospective, and so forth; see Figure 2) leading to extended tasks. Quantitative scheduling is first used in the initial project planning phase, which includes User Story prioritization. Beyond this stage, scheduling becomes a recurring part of the process. For each sprint (iteration), a schedule is computed and used as a benchmark and controlling instrument in the following Review Meeting and Retrospective. The Scrum team experiences can be directly incorporated into the model parameters; then, the next schedule for the succeeding sprint is computed restarting the process until the end of the project.

User Story Prioritization in the Project Planning Meeting

The MRCPS^{SP} model can be used to prioritize the User Stories (superior software modules). A product backlog is created by the product owner, which contains all customer requirements, ideally classified in clear separate modules. For example, an accounting business application can be separated into modules (User Stories) for salary or asset accounting. The Scrum team, including the product owner and ScrumMaster, discuss the User Stories in an upfront project planning meeting and stipulate the MRCPS^{SP} parameters using the AoNG (a possible stipulation process is described later). The product owner then computes a feasible schedule for each User Story receiving four possible sorting rules:

1. Highest Cycle Time (HCT)
2. Lowest Cycle Time (LCT)

Project Class	Redevelopment Projects for Existing Products			New Product Development Projects		
	Scrum	Hybrid approach including MRCPS ^{SP}		Scrum	Hybrid approach including MRCPS ^{SP}	
Parameter/project	Sample Project A	Project 1	Project 2	Sample Project B	Project 3	Project 4
Scrum team	alpha	gamma	alpha	beta	beta	beta
Planned price calculation cost rate	5,000€ (US\$7,000) p.d.	4,400€ (US\$6,160) p.d.	4,400€ (US\$6,160) p.d.	5,000€ (US\$7,000) p.d.	4,400€ (US\$6,160) p.d.	4,400€ (US\$6,160) p.d.
(Maximum) iteration length	30 days	25 days	25 days	30 days	25 days	25 days
Iterations (sprints)	3	3	3	5	4	8
Senior software developers (individual capacity)	2 (N.N.)	1 (4 function points)	2 (4 function points)	2 (N.N.)	2 (8 function points)	2 (4 function points)
Regular software developers (individual capacity)	4 (N.N.)	4 (4 function points)	3 (4 function points)	3 (N.N.)	3 (6 2/3 function points)	3 (4 function points)
User Stories (functional modules)	3	4	3	7	2	10

Note. p.d.: per day; N.N: Nomen nominandum.

Table 2: Project parameters related to the project types (existing and new projects) and the sample projects.

3. Highest Capacity Consumption (HCC)

4. Lowest Capacity Consumption (LCC)

The cycle times can be used to determine the fixed time boxes for the iterations. On the one hand, it is possible to develop the User Stories starting with the most time consuming modules (HCT) with descending order to handle the complex modules first, which can be the longest tasks. On the other hand, the swift programmable modules can be preferred in ascending order (LCT) to build up time buffers. Concerning the capacity consumption, the implicit resource planning of the MRCPS^{SP} can be used. During periods when few programmers are available (holidays, sick days) User Stories with less capacity consumption are favored (LCC). When all staff members are available, the more capacity consuming modules are programmed (HCC).

Iteration Scheduling and Application in Scrum

The MRCPS^{SP} model is intended to deliver quantitative management and decision support, including cycle times, needed resources, and development costs. The computed schedules can be

used as strict targets; furthermore, they serve as a benchmark. Like the entire agile philosophy, the model depends on continuous advancements and fine-tuning; this is achieved by repeated discussion of the model parameters (e.g., activity times, capacities, time lags). Thus, the iteration scheduling meeting is set upfront before the actual development phase (i.e., the iteration). During the meeting, MRCPS^{SP} can be used to determine the next iteration time box. Along with the prioritization from the previous step, the product owner can keep the overview. The iteration is a closed time period in which the development team works without external interference; here, the computed schedule serves as a benchmark or guideline, respectively. The same applies to the short daily meetings. Clearly, meeting the schedule is an ideal situation, but it gives a solid basis for discussions and helps to identify problems early enough to inform the customer. The modeled time lags and buffer activities can be used to give the schedule a natural flexibility. After the iteration review, the retrospective offers additional space for improvements of the quantitative model, for example, by adjusting the basic AoNG. In total, we

vary the existing Scrum methodology and extend it with a quantitative management tool, which can easily be integrated into the flow (see Figure 2). The central beneficiary is the responsible project manager (product owner) who is equipped with schedules to organize the User Stories and to control the project progress; moreover, deadlines can be set and costs managed.

Practical Test and Computational Experiments

Test Scope and Selected Projects

To test our approach we randomly chose three Scrum teams, which were designated as team alpha, team beta, and team gamma. For these teams, we asked the group of product owners to select four typical projects from two kinds of projects (revision of existing software projects and development of new software projects) (Table 2). These four projects should be managed using the new hybrid flow. Furthermore, to guarantee that these projects represented ordinary projects, we compared our results with one historically similar sample project, which was managed by one of the teams with the pure Scrum flow without using the MRCPS^{SP}

A Hybrid Approach to Quantitative Software Project Scheduling Within Agile Frameworks

Project Class	Redevelopment Projects for Existing Products			New Product Development Projects		
	Scrum	Hybrid approach including MRCPS ^{SP}		Scrum	Hybrid approach including MRCPS ^{SP}	
Project	Sample Project A	Project 1	Project 2	Sample Project B	Project 3	Project 4
Cost/key performance indicator	Project cost			Project cost		
Target project costs	300,000€ (US\$420,000)	330,000€ (US\$462,000)	264,000€ (US\$396,600)	650,000€ (US\$910,000)	396,000€ (US\$554,400)	792,000€ (US\$1,108,800)
Total costs	843,000€ (US\$1,180,200)	279,000€ (US\$390,600)	258,000€ (US\$361,200)	1,675,000€ (US\$2,345,000)	464,000€ (US\$649,600)	724,000€ (US\$1,013,600)
Absolute cost deviation	+543,000€ (US\$760,200)	−51,000€ (US\$71,400)	−6,000€ (US\$8,400)	+1,100,000€ (US\$1,540,000)	+68,000€ (US\$95,200)	−68,000€ (US\$95,200)
Relative cost	+281%	84.55%	97.73%	+258%	117.17%	91.41%
Performance (cost)	+181%	−15.45%	−2.27%	+158%	+17.17%	−8.59%
Duration/key performance indicator	Project durations			Project durations		
Target project duration	60 days	75 days	60 days	130 days	90 days	180 days
Total duration	175 days	60 days	63 days	350 days	88 days	158 days
Absolute duration deviation	+115 days	−15 days	+3 days	+220 days	−2 days	−22 days
Project relative duration	+292%	80.00%	105%	+269%	97.78%	87.78%
Performance (duration)	+192%	−20.00%	+5.00%	+169%	−2.22%	−12.22%

Table 3: Results and comparison of project performances related to the project types (existing and new projects) and the sample projects.

model in each project class (sample projects A and B). In Tables 2 and 3 we present parameters based on real company data. Thereby, the general project parameters in Table 2 were modified due to confidential company information settlements and to prevent any lead back to the considered enterprise. The data, thus, represent real projects but allow no conclusion at all to the considered company and its data. Beyond this, the key performance indicators (KPIs) in Table 3 indicate real data.

Projects 1 and 2 represented extensions of existing applications; in contrast, projects 3 and 4 included completely new software. Given parameters were the target costs, which referred to the actual product prices and the target durations, which were derived from stipulated delivery dates. Durations were always measured in work days. To calculate the actual software developer costs in the tested projects, the products of function points and monetary unit (€) per function point were used. Senior developers were evaluated with €1,250 (US\$1,750) and

regular developers with €750 (US\$1,050). Activity durations are mode-dependent (see earlier). The project durations were split up among the ascertained iterations. Thereby, the last iterations were assumed to be short sprints, including the remaining project days. For example, project 2 encompassed 60 days. The first two sprints could theoretically make use of the maximum sprint durations, whereas the final sprint had only a shorter length of 10 days available.

Coordination and Stipulation Process

The Scrum teams preferred to integrate the stipulation process concerning the model parameters and the actual iterations into one flow chart (Figure 3). In this section, we present the procedure, which is an explicit form of the hybrid flow presented in the previous section. The basic idea is that each sprint is planned by the project team and that the team members primarily discuss the given model settings and parameters; thereby, the discussion is

guided by only focusing on the elements used to schedule the sprint. At the end of this coordination process, a sprint schedule is created, which is understood and, more importantly, accepted by each team member. In the following section we briefly describe the process. At first, the sprint increment and hence the goal for the current sprint must be defined. This naturally results from the current project progress or product status, respectively (e.g., which functional module must be developed in the current sprint). Then, the sprint backlog is defined. Note: At this point only basic elements are defined; the technical specifics are designed in the sprint. Typically, the presented AoNG holds for the development process; if adjustments are needed, then the Graph can be changed. Now the basic settings are discussed in the team based on the defined AoNG. The questions are:

1. How long does each activity take for senior developers and regular developers?

parameters. The questions to be answered here are:

1. Which team capacities are measured in function points?
2. What is the sprint budget?
3. How does an activity consume available capacity measured in function points?

Again, the parameters are discussed until an agreement has been achieved. All settings and parameters are detailed enough so that the project team can already account for identified complexity but are still superficial enough to guarantee a clear workflow. The process is kept lean; then, the schedule can be computed and set for the current sprint.

This procedure is repeated after every sprint until the last sprint of the project is finished. At this point, the total costs and durations can be ascertained. By incorporating a quantitative scheduling model, the agile development process becomes more “plan-driven” than without; yet, the computed schedule only provides a work frame including activity finish times and the allocation of developer types to activities. The slight flexibility restriction is overcompensated by capacities and budgets, which are now taken into account.

Computational Results

For the model implementation, the standard software package GAMS-CPLEX was used. GAMS-CPLEX is established modeling software that makes use of a simple syntax and distinct solution procedures to solve, for example, mixed integer linear programs. Due to the small problem instances in the iterative scheduling frame, all calculations were solved in short elapsed time. In Table 3, the results and KPIs are listed. One central result was that in all four projects the computed schedules were sufficient for the practical implementation and did not exceed the upper duration limits. In comparison with similar sample projects, considerably better performances concerning the project

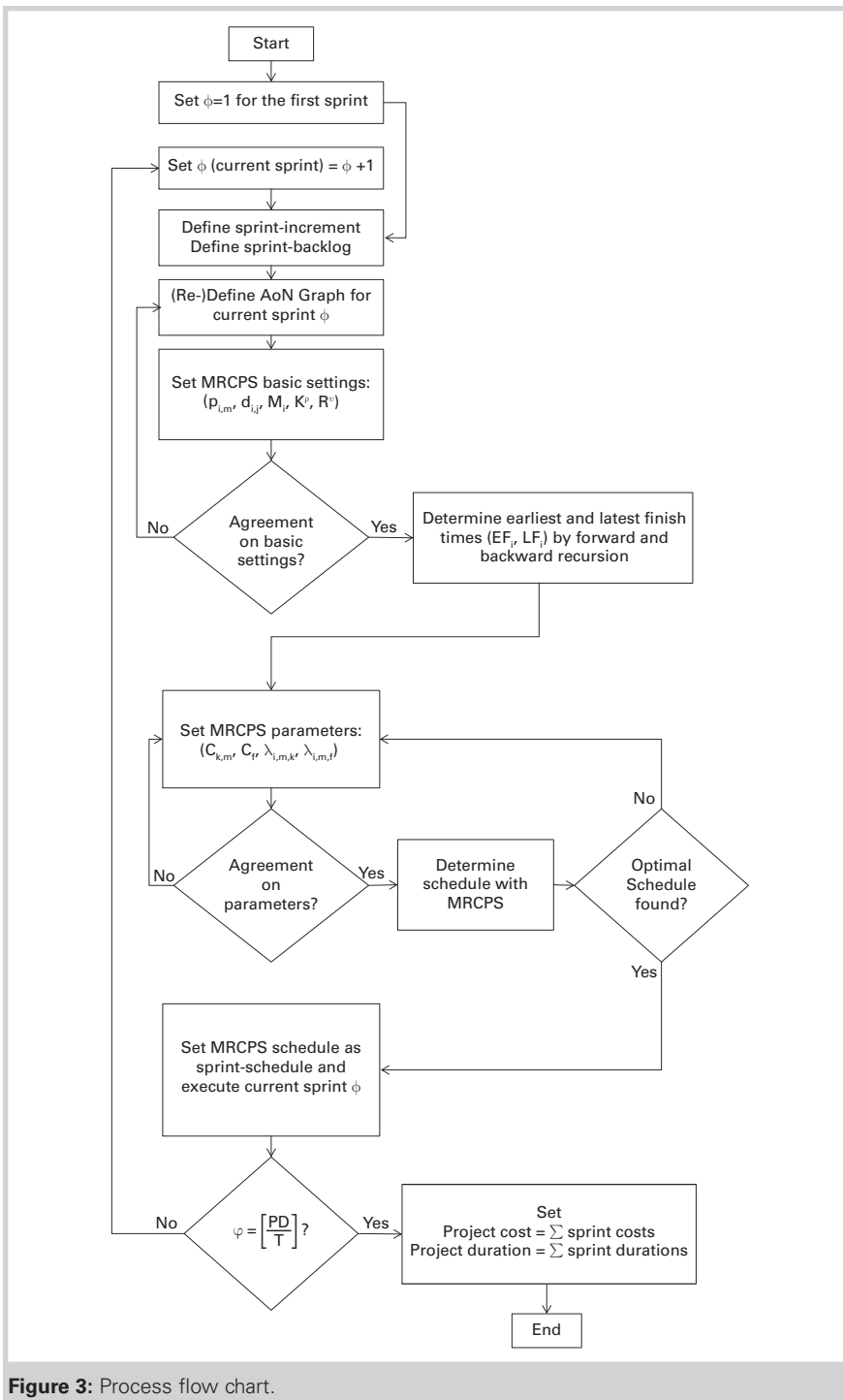


Figure 3: Process flow chart.

2. Should there be time lags as additional time buffers and, if so, between which activities?
3. Are senior developers and regular developers available during the sprint or only one group?

These settings can be discussed until an agreement is found. When an agreement is found, the start and finish times for the MRCPS^{SP} model must be determined. The project team proceeds by discussing the model

A Hybrid Approach to Quantitative Software Project Scheduling Within Agile Frameworks

costs and durations were achieved. This is a clear indication that the proposed approach can also be used for project planning in advance, which was not applied here. The project teams applied a rather safety-oriented strategy and made use of all activities in the AoNG, including the “buffer” activities numbers 6 and 14. In project 2, the project duration was split up into three sprints. The theoretical sprint lengths were 25 work days for the first two sprints and 10 days for the last sprint. The available sprint budgets were €88,000 (US\$123,200) each.

Scheduling with the flow chart procedure in project 2 led to a delivery delay of three days; however, the project costs were less than the available project budget. In contrast, in project 3, the delivery date was met with two additional buffer days but the project costs were considerably higher by 17.17%. This was due to the fact that the objective is to minimize the sprint cycle time; thus, the scheduling tends to allocate faster and hence more cost-intensive resources (senior personnel). An objective focusing on the cost minimization can lead to different results. In this case, management clearly required that the project durations be minimized. On the other hand, projects 1 and 4 showed significant time and cost savings. For example, project 4 could be finished 22 days before the delivery date and project 1 could be completed 15 days before the delivery date, respectively. Both project cost savings compensated for the loss generated in project 3.

Assessment of the Hybrid Quantitative Software Project Scheduling Approach

Company Feedback

After the practical tests, we repeated the interviews with the senior developers, ScrumMasters, and the product owners. Additionally, we also had talks with the Scrum teams who conducted the test projects. For all involved parties the improvements concerning deadlines

and budgets were a clear benefit. Deviating from the general hybrid flow presented earlier and the pure Scrum flow, the software developers preferred to integrate planning activities into the development phase (see Figure 3). This was explained by the fact that planning activities also took time from the predefined available project time, which must be taken into account. Furthermore, it was stated that incorporating a mixed integer linear program into an agile project flow was an unfamiliar approach; specifically, the developers mentioned reservations against a “plan-driven” model. Still, the simplicity of negotiating few clear parameters was positively emphasized. Concerns were expressed regarding the AoNG as the critical basis of the quantitative approach. These concerns focused on the necessity to change or even recreate the AoNG in case new products are incorporated into the business portfolio. For the current two project classes, the developed Graph appeared to be accurate and flexible enough. Adding new activities could lead to impairments. Thus, the responsible company management team decided to launch the presented hybrid flow for projects concerning the existing project portfolio. Projects for new product types should initially be managed with the pure Scrum flow to gather experience for adjusting the AoNG.

Benefits and Limitations

The practical test for the presented hybrid flow showed significant improvements for meeting deadlines and budgets; therefore extending the agile flow by quantitative models proved to be a useful approach. Notably, the benchmarking effect of such quantitative schedules is emphasized. The responsible project management team is equipped with objective and reliable tools to stipulate deadlines. Deviations from the schedule can be used to steer the project and thus take necessary steps; thus adherence to deadlines is significantly supported. Moreover, resources can be planned

as capacities are explicitly considered; however, at this point, the approach can be improved. Measuring development capacities by a function point system proved to be a practical solution in the tests. Nevertheless, the model allocated developers following a mathematical logic. Scarce resources are preferred. Before using the approach, resources must be distinguishable by their costs. For the practical test of our approach, we were able to compare our results using two sample projects. Even though these projects represented the company's common standard, far-reaching statements concerning general performances in software projects cannot be made. In general, the mathematical model is independent from the AoNG; yet, the process modeling must be accurate. The presented AoNG was sufficient for the considered company but a direct transferability to other enterprises is not given, so the approach can only reveal the full effect if the processes are depicted properly.

Summary and Recommendations for Further Research

In this article we focused on quantitative software project scheduling within agile project management frameworks in IT companies. Although traditional sequence-oriented approaches and modern agile methods seemed to be contrary ways, we showed that an agile project flow can be complemented by a quantitative model. We present a modified multi-mode resource constrained project scheduling model for software projects (MRCPS^{SP}) and incorporate it into a Scrum project flow. Within the model, multiple modes representing experts and regular developers and renewable resources as well as non-renewable resources (e.g., budgets) were taken into account. Practical tests based on real data indicated significant improvements. By using this approach, deadlines were met and budgets kept under control. The approach is limited by the underlying process modeling represented by an

Activity-on-the-Node-Graph. To guarantee positive effects, the process depiction must be accurate. Yet, the results indicated that the approach can be used for project scheduling in advance and is only valid for adequate AoN-Graphs. This implies that the software development process can be further improved with additional total project scheduling models. For measuring developer capacities, an estimation based on a modified function points system was used. Although the results proved to be good, further research is indicated to improve this aspect in order to achieve precise depictions. The scheduling was based on work days, which is a rather rough assumption. Therefore, novel process-time scheduling models must be developed that are applicable in the IT industry and enable, for example, (non-)parallelized multi-project, multi-mode scheduling with additional constraints (i.e., [non-]renewable resources and preemption). In our study we were able to show that agile methods and traditional approaches are not contrary worlds. In fact, quantitative models can be important tools to support project management in agile frameworks. The combination of quantitative project management and agile flows is a promising research direction. We believe that hybrid quantitative project management approaches can be a great benefit for software project managers and beyond for other industries. We would welcome an active discussion on this topic.

References

- Abrahamo, S., Poels, G., & Pastor, O. (2006). A functional size measurement method for object-oriented conceptual schemas: Design and evaluation issues. *Software & System Modeling*, 5(1), 48–71.
- Agile Alliance (2001). *Manifesto for Agile Software Development*. Retrieved from <http://www.agilemanifesto.org>
- Albrecht, A. J. (1979). Measuring application development productivity. IBM Application Development Symposium, White Plains, NY: IBM, 83–92.
- Alvarez-Valdes, R., & Tamarit, J. M. (1989). Heuristic algorithms for resource-constrained project scheduling. In R. Slowinski & J. Weglarz (Eds.), *Advances in project scheduling* (pp. 134–143). Amsterdam, Holland: Elsevier.
- Behrouz, A. N., Hojjat, N., & Esmaeil, M. (2012). A branch-and-bound procedure for resource leveling in multi-mode resource constraint project scheduling problem. *Research Journal of Sciences*, 1(7), 33–38.
- Brucker, P. (2002). Scheduling and constraint propagation. *Discrete Applied Mathematics*, 123, 227–256.
- Brucker, P., Drexl, A., Möhring, R., Neumann, K., & Pesch, E. (1999). Resource-constrained project scheduling: Notation, classification, models, and methods. *European Journal of Operational Research*, 112, 3–41.
- Drexl, A., & Gruenewald, J. (1993). Nonpreemptive multi-mode resource constrained project scheduling. *IEEE Transactions*, 25(5), 74–81.
- Frank Cervone, H. (2011). Understanding agile project management methods using Scrum. *OCLC Systems & Services*, 27(1), 18–22.
- Horbach, A. (2010). A Boolean satisfiability approach to the resource-constrained project scheduling problem. *Annals of Operations Research*, 181, 89–107.
- Jalali, S., & Wohlin, C. (2011). Global software engineering and agile practices: A systematic review. *Journal of Software: Evolution and Process*, 24, 643–659.
- Martin, R. (2002). *Agile software development: Principles, patterns, and practices*. Upper Saddle River, NJ: Pearson Education.
- Nachamai, M., Senthil Vadivu, M., & Tapaskar, V. (2011). Enacted software development process based on agile and agent methodologies. *International Journal of Engineering Science and Technology*, 3(11), 8019–8029.
- Reifer, D. J. (2002). How good are agile methods? *IEEE Software*, 19(4), 16–18.
- Salo, O., & Abrahamsson, P. (2008). Agile methods in European embedded software development organisations: A survey on the actual use and usefulness of Extreme Programming and Scrum. *IET Software*, 2(1), 58–64.
- Schwaber, K. (2004). *Agile project management with Scrum*. Redmond, WA: Microsoft Press.
- Steinrück, M. (2011). An approach to integrate production-transportation planning and scheduling in an aluminium supply chain network. *International Journal of Production Research*, 49(21), 6559–6583.
- Steinrück, M., & Jahr, M. (2012). Tactical planning in supply chain networks with customer oriented single sourcing. *International Journal of Logistics Management*, 23(2), 259–279.
- Takeuchi, H., & Nonaka, I. (1986). The new product development game. *Harvard Business Review*, January–February.

Michael Jahr, PhD, is Professor of Business Administration, European University of Applied Sciences (EUFH), Department for Industrial Management, Neuss, Germany. Professor Jahr received his Doctorate in Business Economics from Greifswald University in 2013 and was a Research Associate in the Department of Law and Economics, Greifswald, University, Chair of Business Administration and Entrepreneurship from 2009 to 2011; in 2009, he earned his Master of Science in Economics degree from the University of Cologne. He can be contacted at m.jahr@eufh.de